Simplifying the K215 Correctness Proofs

Uniform decoder plans, semantic interfaces, and smaller proof dependencies

Research report for Vladimir Reshetnikov
ProveIt project

October 21, 2026

Abstract

The most useful next reductions are structural: expose the finite decoder plan before quantifying over its tape interpretation; let simulation consume semantic edge contracts; and separate general set-theoretic lemmas from historical machine endpoints. This packet supplies a new Lean-checked strengthening of the actual K215 decoder theorem: one fixed clock and primitive-entry port work for every gap and register list. A checked source-depth lemma proves that every such prefix makes at least two physical steps. The complete 23-module decoder/runtime dependency closure and both new modules were freshly compiled. The packet also supplies a focused, independently checked Coq port of Lean's existing 58-step abstract implication proof, two axiom-free Lean enumerator abstractions, and an uncompiled import-refactoring patch with a reproducible net reduction of 13 local modules and 233,503 source bytes in the projected K215 dependency closure. These changes preserve the accepted 215-state machine. Production endpoint integration and the internalization of its external correctness theorem are explicitly distinguished from the checks completed here.

Reviewed baseline. All repository-specific findings refer to commit

`2ca3a5c16f0876d2cab842056353c03a1ed2f5f0`

The exact dense transition table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`

No production repository file was modified during this investigation. The new files and the proposed patch are delivered for review and incorporation.

Contents

1 Recommendation and status **3**

2 What the current repository already establishes **3**
2.1 The exact theorem and the internalization objective . . . 3
2.2 Avoid repeating completed simplifications . . . 4

3 Checked result: choose the decoder plan before interpreting it **4**
3.1 The unnecessary dependency in the old statement . . . 4
3.2 The strengthened source theorem . . . 5
3.3 Checked positivity from three levels of the source . . . 5
3.4 The exact zipper theorem . . . 6
3.5 What was compiled . . . 6

4 Use the shared prefix through semantic coverage **6**
4.1 The interface that currently prevents reuse . . . 6
4.2 A conservative migration . . . 7
4.3 Why the converse halting implication is preserved . . . 7
4.4 Reproducible decoder work census . . . 7

5 Separate model theory from historical program endpoints **8**
5.1 Three overly broad dependency routes . . . 8
5.2 Exact scope of the projected reduction . . . 9

6 One abstract implication proof in both assistants **9**
6.1 What to share . . . 9
6.2 Why propositional automation is not a substitute . . . 10
6.3 Validation and the real tradeoff . . . 10

7 Reusable enumerator contracts and finite progress **11**
7.1 Early halt is an allowed result, with a separate soundness invariant . . . 11
7.2 A scheduler theorem without an unexplained fairness assumption . . . 11

8 Integration plan and acceptance boundaries **11**

9 Archive contents and reproduction **12**

1 Recommendation and status

The final K215 theorem is already short. Its Lean endpoint is a small application of a shared endpoint theorem; the current enumerator seed theorem is likewise a one-line instance of a general result. Further gains should come from reducing the proof's dependencies and eliminating repeated computational work below those interfaces. Trying merely to compress the final theorem statement would obscure the useful boundaries. The relevant current sources are `KEndpoint215`, `SeedPerm`, and the maintained verification page [2, 1].

The recommended order is:

1. Adopt the stronger uniform decoder plan and its exact zipper adapter. This is a completed local proof development against the current source and table.
2. Introduce semantic edge coverage, with the current Boolean checker as its first producer. Then use the shared decoder plus residual tail checks for one existing site batch.
3. Separate general model constructions from the old 254-state endpoint modules. The provided patch is a concrete source proposal with quantified dependency savings.
4. Give Rocq the same abstract implication interface as Lean, or select the already available smaller numerical rendering if source size is the sole objective.
5. Share early-halting composition and finite candidate-search lemmas across the enumerators. The generic lemmas are checked; their concrete instances remain to be connected.

Deliverable	Result	Validation in this packet
Uniform decoder and zipper	One clock and port before all tape/gap/register parameters; at least two physical prefix steps	Lean 4.32.0; 23 original modules plus 2 new modules
Abstract implication core	Same 58-step derivation and interface in Rocq	Coq 8.18.0; closed reports and independent `coqchk`
Numerical implication adapter	Preserves `DomImp` and `rho8_all`	Focused Coq 8.18 check with documented import-name compatibility change
Enumerator proof rules	Composition of partial-result contracts; finite candidate progress	Two standalone Lean files, ten axiom-free exported results
Import refactoring	180 to 167 BusyBeaver-local modules in projected closure	Source-graph measurement and patch applicability; integrated build pending
Semantic coverage and tails	Reuse decoder proof in production site coverage	Precise implementation proposal; full site migration pending

Table 1: Different validation scopes are deliberately reported separately.

The report does not claim a new full endpoint build, a smaller transition table, or a measured production speedup. The new decoder theorem and the standalone proof artifacts are actual checked results; the integration proposals have their own outstanding obligations.

2 What the current repository already establishes

2.1 The exact theorem and the internalization objective

The accepted machine has 215 working states, two tape symbols, a two-way infinite tape, blank input, initial state zero, and a separate halt. Initialization is included in that state count. The repository records checked external theorems of the form

$$\mathrm{Halt}(K215, \mathrm{blank}) \quad \Longleftrightarrow \quad \mathrm{Prov}_{\mathrm{ZFC}}(\bot).$$

Negating both sides gives nonhalting if and only if syntactic consistency. Consistency is not assumed in either direction. The repository's verification page explicitly distinguishes these external proof-assistant results from an internally encoded PA proof or a derivation in ZFC's own proof syntax [1].

This distinction matters to the stated research objective. An external Lean or Rocq theorem about a ZFC proof predicate is valuable evidence, but it is not automatically the separately encoded object

$$\mathrm{ZFC} \vdash (\neg \mathrm{Halt}(K215, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})).$$

The present work simplifies the available proof architecture. It does not supply that additional internal derivation. The finite clock construction and explicit derivations proposed here also make a future arithmetization more transparent, but that is an architectural benefit, not a completed formalization claim.

2.2 Avoid repeating completed simplifications

The reviewed branch already contains three especially relevant improvements. First, Lean's `DominatedImplication.lean` contains a 58-step condensed-detachment derivation of unrestricted implication row eight, composition, and prefixing. It is representation independent and its public theorems have no axioms [7]. Second, `KHome` and `KHomeReady` already share the scanner's return-to-dispatch proof. The readiness gate checks fifteen scan rows and an eighteen-step boundary; the shortcut has positive physical progress [3, 4]. Third, the shared decoder and its exact source and zipper bindings already exist. They are auxiliary to the endpoint, while the maintained `checkEdge` still starts from the full source-path configuration [5, 6].

The phase-nine record distinguishes a 192-module endpoint closure from a 197-module union including five decoder-prefix modules. It records 984.608 seconds for the nine fresh site batches and 1,649.340 seconds in historical phase-six records. That is a historical comparison, not a new paired benchmark in this investigation [3]. The original full build and the later incremental checkpoints also have different module counts. These numbers must not be mixed with the local import-graph census in Section 5.

3 Checked result: choose the decoder plan before interpreting it

3.1 The unnecessary dependency in the old statement

The decoder's source word determines a finite route through a role graph. Every edge on that route names a block with a fixed duration. Its terminal edge determines a primitive-entry state and the final source bit. These choices depend on the word and the certified finite graph; they do not depend on tape values outside the word, the gap length, or the register list.

The existing `source_path_chain_routed` nevertheless produces its clock and exit port inside the universal quantifiers over tape and head position. In simplified form, its interface is

$$\forall a, t, \quad \mathrm{Input}(w, a, t) \Longrightarrow \exists N, q, b, \quad \mathrm{Run}(w, N, q, b, a, t).$$

The later `entry_uniform` selects a port using a particular gap/register instance, then identifies ports from other instances through route and suffix uniqueness lemmas. It leaves the exact clock existential inside the parameter quantifiers [5].

There is no general logical rule that exchanges these quantifiers. The right repair is a stronger induction that constructs the plan from the source path first. This is the new theorem in `proofs/UniformDecoder.lean`.

3.2 The strengthened source theorem

Let w be a source word, and suppose it describes a valid source-DAG path from the K215 root to a primitive. Write $\mathrm{Reads}(t, a, w)$ for the assertion that the word is present on tape t, starting at integer address a. Write run_M^N for the exact partial execution after N physical steps.

Theorem 3.1 (Uniform K215 decoder plan)**.** *For every such source path, there are natural numbers N, q and a bit b, chosen independently of a, t, such that*

$$|w| - 1 \le N \le 7(|w| - 1),$$

the corresponding `RoutedEntry` *certificate holds, and for every head position $a \in \mathbb{Z}$ and full tape t,*

$$t(a-1) = 1 \ \wedge \ \mathrm{Reads}(t, a, w) \quad \Longrightarrow \quad \begin{array}{l} \mathrm{run}_M^N(84, a, t) = \mathrm{some}(q, a + |w| - 1, t), \\ t(a + |w| - 1) = b. \end{array}$$

The whole tape is preserved. No finite-support hypothesis is imposed on the full tape.

Proof. The proof generalizes over the role identifier and its matching source node, then inducts on the source path. A source leaf cannot be the current role node because the role's checked edge specification requires a fork.

For a primitive edge, the path has one remaining bit. Select $N = 0$, the current role state, and that bit. The route certificate is already finite data. For every matching tape the chain is reflexive and the last-bit assertion is exactly the first `Reads` hypothesis.

For an internal edge, the checked specification selects a successor role and a block B. Apply the induction hypothesis to the successor before introducing a, t, obtaining a fixed tail clock N', exit state, and final bit. Select $N = B.\mathrm{steps} + N'$. Construct the extended route certificate immediately. Only then introduce the tape. Its `Before` and `Reads` hypotheses provide the block window, and the existing framed-block theorem preserves the surrounding tape. Compose that block with the already selected tail chain.

The bounds follow from $1 \le B.\mathrm{steps} \le 7$. The tail word is nonempty, as witnessed by its terminal-bit split, so natural subtraction in $|w| - 1$ is handled without an implicit positivity assumption. All choices occur in the finite structural part of the argument; no choice principle is needed to move the existential witnesses. □

`source_path_chain_uniform` is the abstract result. The theorem `reaches_primitive_uniform` is the literal K215 instance. The assumption reports for the generic result contain `propext` and `Quot.sound`. The concrete instance also inherits `Classical.choice` from the existing finite-check infrastructure. The claim that witnesses are selected structurally is therefore not a claim that every imported theorem in the concrete dependency closure is axiom-free.

3.3 Checked positivity from three levels of the source

The literal source root is node 201, with children 16 and 200. Those have children $(2, 15)$ and $(32, 199)$, respectively; all four of those nodes are forks. Consequently every complete source word has at least three bits.

The new file defines a small Boolean `atLeastDepth` inspection and proves its generic soundness by induction: depth zero is vacuous; positive depth excludes leaves and decreases through the selected fork child. The exact depth-three fact is checked by `decide +kernel`. Therefore

$$\mathrm{SourcePath}(w) \Longrightarrow 3 \le |w| \Longrightarrow 2 \le N.$$

The positivity theorem requires no extra caller-supplied depth premise. This is a kernel-checked statement about every source path, not an inference from a finite run sample or a trusted assertion supplied by Python.

3.4 The exact zipper theorem

The second new file, `proofs/UniformZipper.lean`, connects the uniform plan to the repository's existing zipper semantics and literal expanded source. Its result is

$$\begin{aligned}\exists\, \mathrm{pre}, b, q, \mathrm{op}, \mathrm{arg}, N: \quad & w = \mathrm{pre} +\!\!+ [b], \quad e = \mathrm{primitive}(\mathrm{op}, \mathrm{arg}),\\ & \mathrm{RoutedEntry}(w, q, b, \mathrm{op}, \mathrm{arg}), \quad 2 \le N, \quad |\mathrm{pre}| \le N \le 7|\mathrm{pre}|,\\ & \forall n, \mathrm{vs}, \quad \mathrm{run}_M^N(\mathrm{conc}(n, \mathrm{vs}, \mathrm{start}(w))) = \mathrm{some}\big(\mathrm{conc}(n, \mathrm{vs}, \mathrm{leafPort}(\mathrm{pre}, b, q))\big).\end{aligned}$$

The clock is outside $\forall n, \mathrm{vs}$. The theorem has no register-validity or nonempty-register-list premise because it stops before the primitive action. Its proof uses the existing whole-tape-to-zipper reflection and preserves the complete reversed prefix, marker, gap, and register frame.

`walk_prefix_uniform` supplies the same result for the actual interpreter walk, using `consume P d`. This retains implicit zero padding; substituting `P.take d` would change the contract. `prefix_tail_sound` then composes the positive prefix with any valid ordinary-reachability tail. The old uniqueness lemmas are not needed by this new construction. Removing them from production is a later API decision: existing external callers must be checked, and compatibility wrappers may be preferable.

3.5 What was compiled

A fresh serial Lean 4.32.0 build compiled the 23 original modules reachable from `KDecoderPrefix215`, followed by the two new modules. All 25 succeeded. The actual literal decoder, table checks, source binding, symbolic runtime, and old simulation theorem were compiled in that closure; no placeholder model was substituted. The final logs and per-source SHA-256 values are in `evidence/lean-uniform/`.

The new leaf and walk theorems report only the same three standard Lean axioms: `propext`, `Classical.choice`, and `Quot.sound`. There is no `sorryAx`. This is a local checked extension of the production semantics, not a new full build of the ZFC endpoint or a completed adoption of the new site producer.

4 Use the shared prefix through semantic coverage

4.1 The interface that currently prevents reuse

The current `KSim.Covered M p` says that every walk reaches a leaf whose `checkEdge M Q e` is true. That Boolean checker executes from `start Q` through the whole path and primitive. The simulation proof repeatedly unpacks the Boolean expression to recover the semantic facts it actually needs: path length, register bounds, branch preconditions, exact register effects, and positive reachability [6].

Define a semantic proposition `EdgeRealizes M Q e` containing those facts directly. Its length field is the current $|Q| \le 48$ guard. Increment and decrement also retain $r < 9$. Each live branch universally quantifies the natural gap and register list, retains its `Valid` premise, and reaches the identical `tgtConf` expression currently used by `checkEdge`. The halt branch asserts actual halting, and a `node` edge is impossible.

For example, the increment case requires, for every valid register frame,

$$\mathrm{conc}(n, \mathrm{vs}, \mathrm{start}(Q)) \longrightarrow^{+} \mathrm{conc}(n, \mathrm{vs}, \mathrm{tgtConf}(\mathrm{incPath}(Q), \mathrm{allOnes}(Q), \mathrm{inc}(\mathrm{physOf}(r)))).$$

The decrement case includes both the zero and nonzero branch contracts. Break and continue retain their existing ancestor-path transformations, overflow conditions, and targets. A semantic interface must not silently discard any of those distinctions.

Then set

$$\text{SemanticCovered}(M,p) := \forall P,\ \exists e,d,\quad \text{walk}(p,P) = \text{some}(e,d)\ \wedge\ \text{EdgeRealizes}(M,\text{consume}(P,d),e).$$

The omitted walk arguments here are exactly the current node bound, root, and initial depth; the implementation should preserve them literally.

4.2 A conservative migration

First prove `checkEdge_sound` using the existing `checkEdge_length`, `runTo_sound`, and `runHalt_sound`. The existing Boolean `Covered` then implies `SemanticCovered`. Refactor `sim_step` and `sim_halt` to use that interface. The register arithmetic, spare-list extension, path cleanliness, and overflow proofs are retained. `sim_stop` still requires total walk coverage and the exclusion of node leaves. Preserve the public `halts_iff` as a wrapper. At this stage all existing clients can continue using exactly the old certificates.

Next add a second producer: use the shared prefix theorem to reach `leafPort`, then run a residual checker to the unchanged full target. Initially retain every source occurrence and both decrement outcomes. This is a smaller step than proving all parameterized primitive families at once. A mixed rollout can choose the old full check or the new prefix-plus-tail proof for each occurrence.

For the residual checker, initialize the executor's `moved` flag to true. Its soundness provides ordinary reachability from the primitive-entry port; the prefix supplies the needed physical progress:

$$s \longrightarrow^{+} p \quad\wedge\quad p \longrightarrow^{*} t \quad\Longrightarrow\quad s \longrightarrow^{+} t.$$

The existing `Reach1.trans_reach` proves the composition, and the new `prefix_tail_sound` specializes it to the uniform decoder contract. A zero-length tail is harmless here. This argument depends on positive prefix progress; it must not be exported to a source with a one-bit leaf without an explicit progress premise.

4.3 Why the converse halting implication is preserved

One must preserve positive *physical* progress for every live source step. In the existing deterministic simulation, if the source takes a step, the target makes at least one step and reaches a related configuration. A finite halting target run therefore has a strictly smaller remaining time after a simulated source step. Strong induction on that time yields the reverse halting implication. A proposed macrostep that could silently loop through zero-duration administrative work would not support this induction.

The new interface does not alter that proof. It changes the producer of its premises. The full prefix retained by `leafPort` also matters: two predecessor bits suffice for the checked internal decoder windows, but they do not summarize every primitive return context. Break and continue depend on ancestor boundaries; overflow can move the marker. Generalizing primitive families is appropriate only after their complete frame contracts have been proved.

4.4 Reproducible decoder work census

`code/decoder_prefix_census.py` independently parses the pinned decoder and transition table, checks all 281 local block traces, checks primitive transition agreement, traverses every source/role occurrence, and literally simulates each full decoder prefix. It rejects reads outside the specified finite window and requires restoration of every cell. It imports no repository generator. The result and source hashes are in `evidence/decoder_prefix_metrics.json`.

Quantity	Measured value
Expanded source leaves	4,751
Branch outcomes, counting both decrement cases	6,176
Source DAG nodes / decoder roles / distinct blocks	202 / 234 / 281
Exposed primitive-role interfaces	97
Source-word lengths	3–47
Exact decoder-prefix clocks	2–55
Sum of prefix clocks, once per leaf	165,876
Sum of prefix clocks, once per branch outcome	213,945
Distinct local block traces, checked once each	353 physical steps
Internal edge occurrences / their trace-step sum	4,749 / 5,721

Table 2: Finite decoder-only evidence. The clocks and counts do not predict kernel time.

The 213,945 figure quantifies physical prefix work that is repeated in a full branch-by-branch interpretation. It is not a symbolic-executor instruction count, kernel reduction count, or estimated speedup percentage. The minimum depth and positive progress also have the separate kernel proof in Section 3. The exact maximum of 55 in this table remains finite census evidence; the checked uniform theorem states the structural upper bound $7(|w| - 1)$.

The largest repeated primitive families are increment 1 at state 57/read 0 and continue 2 at state 57/read 1, each with 267 occurrences; the analogous increment 6 and continue 3 families at state 62 each occur 257 times. These are useful later targets. The already designed 86-occurrence CB1 case may still be the easiest first parametric pilot. Occurrence count alone does not establish a correct generic primitive theorem or determine its proof difficulty.

5 Separate model theory from historical program endpoints

5.1 Three overly broad dependency routes

The current K215 import closure contains the historical 254-state enumerator and register-program proof even though the K215 endpoint consumes more general semantic results. The key paths include

$$\begin{aligned} \texttt{ConstructibleChoice} &\to \texttt{ZFNative} \to \texttt{NativeTheory254} \to \texttt{L2Root254}, \\ \texttt{KQ1ZF} &\to \texttt{WFTheory} \to \texttt{WFModel} \to \texttt{QuotientZF0} \to \texttt{Enum254}, \\ \texttt{L2Lib222} &\to \texttt{Enum254}. \end{aligned}$$

The first consumer needs `ZFNative.models_ZF`. The second needs the generic well-founded-part model construction, especially `WFModel.domain_models_ZF`. The third uses the underlying numerical code layer. These module dependencies are wider than those uses [9].

The supplied patch moves `models_ZF` verbatim to `ZFModels.lean` and moves the generic prefix of `WFModel.lean` verbatim to `WFModelCore.lean`. The original files import the new cores and retain their historical specializations, with declaration names preserved. Consumers import the narrower modules. `KQ1ZF` explicitly imports `InductiveAttempts`, retaining the infinity lemma that previously arrived transitively. `L2Lib222` imports `Code254` directly; its maintained generator receives the matching substitution.

This is not deletion of the old mathematics. It keeps those theorems available for their original callers while removing them from an unrelated endpoint's build chain. The proposed files, patch, and generator are provided under `patches/` and `code/import_boundary_proposal.py`.

5.2 Exact scope of the projected reduction

The census counts modules physically under `Computability/BusyBeaver/Lean` and treats seven external library names as boundary nodes. The boundary set is unchanged. It does not claim to reproduce the verification page's 192-module project-wide closure count.

Scoped endpoint closure	Baseline	Proposed graph	Net reduction
BusyBeaver-local modules	180	167	13
Source lines	36,261	32,296	3,965
UTF-8 source bytes	1,989,076	1,755,573	233,503

Table 3: Static source-graph projection, including the new core modules.

Fifteen historical modules leave the closure and two core modules enter it. The net byte reduction is approximately 11.74% in this measured scope. These are source-size and dependency measurements, not proof-term or elapsed-time measurements.

The patch passes `git apply -check`. The generator checks byte-identical extraction of the moved declaration regions and records source hashes. The integrated patch has not been compiled. In Lean, narrowing imports can remove unqualified names, instances, attributes, or simplification lemmas; a textual use analysis cannot settle every elaboration dependency. Acceptance therefore requires an affected-module build, the two unchanged K215 endpoint clients, and the legacy clients of the compatibility wrappers. This is a bounded, concrete proposal with a substantial potential benefit, not a replacement for that kernel check.

6 One abstract implication proof in both assistants

6.1 What to share

The numerical row is

$$\rho_8(a,b,c,u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)).$$

The dominated calculus initially grants it only under $u \preccurlyeq a$. Lean already derives the unrestricted row using a representation-independent 58-step proof. The production Rocq `DomRho8.v` at this baseline still contains the 22,673-byte compact certificate replay, while `rho8cd` also contains an 11,190-byte, 52-step numerical rendering proposal [7, 8].

The new `rocq/DominatedImplication.v` ports the current abstract Lean derivation to Rocq. Its interface needs only an arbitrary carrier A, a binary operation $\Rightarrow$, a relation $\preccurlyeq$, and a predicate T, with:

$$\begin{aligned}
&x \preccurlyeq x,\\
&u \preccurlyeq x \Longrightarrow u \preccurlyeq (x \Rightarrow y), \qquad u \preccurlyeq y \Longrightarrow u \preccurlyeq (x \Rightarrow y),\\
&T(a \Rightarrow b) \wedge T(a) \Longrightarrow T(b),\\
&u \preccurlyeq a \Longrightarrow T(\rho_8(a,b,c,u)).
\end{aligned}$$

It proves unrestricted `rho8_all`, composition, and right prefixing. It does not assume transitivity or antisymmetry of the relation, injectivity of the operation, a formula decoder, arithmetic, a truth constant, or a logical interpretation of codes. In particular it never needs the numerical equation `wim 0 0 = 0`.

The numerical adapter preserves the existing `DomImp` record, including its truth field, and the exact public `rho8_all` statement. It instantiates the growth laws using the established pairing bounds.

Preserving the unused record field keeps the interface compatible while allowing the mathematical core to remain smaller.

6.2 Why propositional automation is not a substitute

Here $\Rightarrow$ is an operation on codes and T is an arbitrary derivability predicate. A propositional tautology prover would prove something about logical implication, not about membership in T. Identifying those two levels would require another soundness or interpretation theorem. Explicit applications of modus ponens avoid this obligation and remain valid for noninjective encodings and junk codes.

The 58 ordinary proof lemmas apply earlier results at explicit substitution instances. There are 33 discharged guard occurrences: 15 reflexive, 5 supplied by local hypotheses, and 13 established by short left/right operand paths. The latter paths are R nine times, RR twice, LL once, and LLR once. Both growth laws are used; these counts do not justify deleting either law.

6.3 Validation and the real tradeoff

The port and numerical adapter were compiled with Coq 8.18.0. The generic results and numerical `rho8_all` print closed assumption reports, and independent `coqchk` checks returned exit zero. For the numerical adapter, the sole version-compatibility source change was `From Stdlib` to `From Coq` in the arithmetic import. The two archived numerical dependencies are hash pinned. The modern source, compatibility patch, and receipts are supplied separately.

This is not a production Rocq 9.1.0 endpoint build. Replaying the current exact Lean abstract core under Lean 4.32.0 also succeeded, with all three public results axiom-free. The mathematical derivation is inherited from the repository; the contribution here is the common Rocq abstraction and an independent explicit-instance audit.

Rocq proof source	Bytes
Current production replay	22,673
New generic core	14,655
New numerical adapter	1,389
Generic core plus adapter	16,044
Existing numerical 52-step proposal	11,190

Table 4: Two different choices: the common abstract interface or the smallest numerical file.

The proposed shared version saves 6,629 bytes, or approximately 29.24%, against the current production replay. It is larger than the existing numerical 52-step proposal. Its advantage is sharing the same mathematical interface and derivation with Lean. The row-only dependency slice of the abstract derivation contains 52 lemmas; the six additional helpers serve composition and prefixing. Whether to retain the full common interface or specialize the Rocq consumer should be an explicit maintenance choice.

The included `rocq/check_and_render.py` checks an expanded certificate containing explicit substitution arguments. It performs no unification or proof search: it checks backward references, substitution scopes, exact MP equations, guard witnesses, and the three intended public goals. Five malformed controls are rejected. The generated ordinary proof is subsequently checked by the kernel, so the Python checker is not part of the logical trusted base. Its JSON and code are review artifacts and are not counted as proof-source savings.

7 Reusable enumerator contracts and finite progress

7.1 Early halt is an allowed result, with a separate soundness invariant

The current command-block contract says that a finite block either halts early or returns a specified state. Abstract its repeated case split as

$$\mathrm{Returns}(r, x) :\Longleftrightarrow r = \mathrm{none} \ \vee \ r = \mathrm{some}(x).$$

For option composition, the two basic rules are

$$\mathrm{Returns}(r, x) \wedge \mathrm{Returns}(f(x), y) \Longrightarrow \mathrm{Returns}(r \ \mathrm{bind} \ f, y),$$
$$\mathrm{Returns}(r, x) \wedge f(x) = \mathrm{none} \Longrightarrow r \ \mathrm{bind} \ f = \mathrm{none}\,.$$

Both follow by the two possible results of r. List concatenation versions follow from `exec_append`. The included `proofs/OptionContracts.lean` proves these rules and prefix variants using only `Init`. All eight printed exported dependency reports are axiom-free under Lean 4.32.0.

The intended application is to `gen_contract`, `mp_contract`, and `program_none`: rewrite the existing execution through `cexec_append`, apply the generic composition rule, and prove only the concrete suffix behavior. For modus ponens, apply the two-prefix rule to its two compiled subproofs. The current `EnumPerm` proofs are already short; the principal benefit is sharing the pattern with the older and future command encodings [10].

In this application `none` represents an actual halting event reached during finite execution. It does not mean divergence. The permissive contract alone is not a soundness theorem: an erroneous block that always halts satisfies it. The global invariant proving that every halt yields a refutation must remain in the proof.

7.2 A scheduler theorem without an unexplained fairness assumption

Suppose $\mathrm{Ready}(s)$ means that execution is between candidates, and $\mathrm{next}(s)$ is the next candidate counter. The concrete fetch/drain proof should establish

$$\mathrm{Ready}(s) \Longrightarrow (\exists t, \ s \longrightarrow^* t \wedge \mathrm{Ready}(t) \wedge \mathrm{next}(t) = \mathrm{next}(s) + 1) \ \vee \ \mathrm{Halts}(s).$$

Finite reachability must be reflexive and transitive, and halting must propagate backward along it. For an initial state s satisfying $\mathrm{Ready}(s)$, induction on k then proves that counter $\mathrm{next}(s) + k$ is finitely reached or the original state already halts.

If a compiled reset-and-refute block has a sufficiently late candidate code, its contract provides a k for which every ready state at that counter halts. Compose that fact with the finite-progress theorem to obtain halting from the original state. `proofs/FairSearch.lean` proves both statements with no axioms and only `Init`. Its name does not hide an infinite fairness assumption: all progress premises are finite and must be discharged by the concrete enumerator.

The fetch, drain, command encoding, stack invariants, and killer-candidate construction remain substantive concrete obligations. The generic theorem shares their assembly; it does not make those obligations assumptions of the final machine endpoint.

8 Integration plan and acceptance boundaries

First change: uniform decoder information. Incorporate the stronger source-path theorem and literal depth-three fact. Replace the later uniformity reconstruction by the checked zipper adapter. Keep older public statements as consequences if they have callers. This makes progress and clock uniformity explicit before changing the production site-checking strategy.

Second change: semantic coverage. Introduce the semantic edge record and an adapter from every existing Boolean site check. Rebuild the unchanged simulation consumers before introducing a new producer. Then route one current site batch through exact prefix plus residual tail, retaining the same table, targets, branch cases, and register preconditions. Compare this batch against the old producer on the same toolchain and machine. Only after that comparison should a production speedup be stated or the remaining batches migrated.

Third change: narrower module imports. Apply the supplied patch in an isolated checkout. Rebuild the changed model cores and their descendants, then check both endpoint clients and the historical wrappers. Regenerate the L2 source to verify that the import improvement survives its producer. Refresh the source-bound verification record after these builds succeed.

Fourth change: common Rocq implication core. Compile the generic file and the numerical adapter with the repository's production Rocq 9.1.0 and stdlib configuration. Update the generator, module order, and manifest to reproduce the new files. Compile `ThmDom` and its actual dependents, then run the recursive endpoint check before declaring the new lane adopted. The current checked replay remains a reproducible baseline during this process.

Fifth change: enumerator instances. Connect the option-contract rules to the existing finite execution semantics and instantiate the scheduler from the current fetch/drain lemmas. The existing seed theorem already has a small public wrapper; aim to reduce duplicated compiler and scheduler reasoning rather than changing that wrapper's statement.

Each change has a separate acceptance point. This is particularly useful here because the repository already records merges between independently developed proof lanes. A successful historical check belongs to its exact source bytes and imported definitions. A new producer, relocation, or executable-definition change should not inherit an old check merely because a final theorem statement is unchanged.

9 Archive contents and reproduction

Path	Purpose
`report.tex`, `report.pdf`	Complete report and compiled document.
`proofs/UniformDecoder.lean`	Strong source-plan theorem; depth check and positivity.
`proofs/UniformZipper.lean`	Literal source leaf/walk adapters and prefix-tail composition.
`proofs/OptionContracts.lean`, `proofs/FairSearch.lean`	Standalone checked enumerator proof abstractions.
`rocq/`	Generic implication core, numerical adapter, explicit certificate, checker, compatibility patch, and focused receipts.
`patches/import_boundaries.patch`	Reviewable import refactoring; integrated compilation pending.
`patches/proposed/`	Proposed new/edited files for inspection.
`code/`	Reproduction of the decoder checks, proof build, and import census.
`evidence/lean-uniform/`	Final 25-module source-bound build record and raw outputs.
`evidence/`	Decoder census, import projection, standalone proof logs, and source review.

Path	Purpose
`README.md`, `REPRODUCE.md`	Status map, build commands, and remaining integration obligations.
`NOTICE.md`, `LICENSE`	Attribution and inherited license boundary.
`MANIFEST.sha256`	Artifact integrity inventory, excluding itself.

The Lean decoder build takes a checkout at the pinned commit and a Lean 4.32.0 executable. It discovers and freshly compiles the 23 original dependencies, then the two new files, recording each source hash and compiler exit status. It needs only the bundled Lean library for this local closure, not a full mathlib or ZFC-endpoint build. The standalone enumerator lemmas likewise require only `Init`.

The Coq core uses the default prelude. Its numerical adapter pilot uses two pinned arithmetic source modules archived in the repository; the reproduction script applies the explicit import-name compatibility change for Coq 8.18. The production Rocq 9.1 adoption should use the modern adapter source and its actual maintained dependencies.

All source/proof artifacts are text. Compiled Lean and Coq objects and downloaded toolchains are omitted from the archive. The final build records provide their hashes, commands, logs, and validation scope. The report's recommendation is therefore concrete: adopt the newly checked uniform decoder information first, expose semantic coverage so it can be used, and reduce unrelated dependencies without changing the accepted machine or weakening its correctness statement.

References

[1] ProveIt, *Formal correctness of the exact 215-state machine*, pinned verification page. Repository source and recorded assurance boundary.

[2] ProveIt, `KEndpoint215.lean` and `SeedPerm.lean`, pinned sources.

[3] ProveIt, *Ninth K215 proof-simplification checkpoint*. Phase-nine status and timing evidence.

[4] ProveIt, `KHome.lean` and `KHomeReady.lean`, shared return proof and gate.

[5] ProveIt, `DecoderPaths.lean`, `DecoderSourceBinding.lean`, `KDecoderPrefix215.lean`, and `K215Decoder.lean`, pinned decoder sources.

[6] ProveIt, `KSim.lean` and `KRuntimeCore.lean`, coverage and simulation.

[7] ProveIt, `DominatedImplication.lean`, `DomRho8.lean`, and `rho8cd` derivation and producer notes. The implication derivation is inherited mathematical work.

[8] ProveIt, production `DomRho8.v`, `Source215.v`, and lane-specific adoption status.

[9] ProveIt, `ZFNative.lean`, `WFModel.lean`, `KQ1ZF.lean`, `ConstructibleChoice.lean`, and `L2Lib222.lean`, source of the import projection.

[10] ProveIt, `EnumPerm.lean` and `EnumDom.lean`, compiler contracts and candidate enumeration.

[11] ProveIt, K215 attribution notice. Inherited mathematics and machine ancestry remain credited to their original contributors, including the ProveIt contributors, Elliot Glazer, Jeff Ketchersid, and the other authors identified in those notices.